Chroma

Querying Collections › Metadata Filtering

Querying Collections

Metadata Filtering

Copy page

Learn how to filter query results by metadata in Chroma collections.

The `where` argument in `get` and `query` is used to filter records by their metadata. For example, in this `query` operation, Chroma will only query records that have the `page` metadata field with the value `10`:

Python TypeScript Rust

```
collection.query(
    query_texts=["first query", "second query"],
    where={"page": 10}
)
```

In order to filter on metadata, you must supply a `where` filter dictionary to the query. The dictionary must have the following structure:

Python TypeScript Rust

```
{
    "metadata_field": {
        <Operator>: <Value>
    }
}
```

Using the `$eq` operator is equivalent to using the metadata field directly in your `where` filter.

Chroma

```
}

# is equivalent to

{
    "metadata_field": {
        "$eq": "search_string"
    }
}
```

For example, here we query all records whose `page` metadata field is greater than 10:

Python TypeScript Rust

```
collection.query(
    query_texts=["first query", "second query"],
    where={"page": { "$gt": 10 }}
)
```

Using Logical Operators

You can also use the logical operators `$and` and `$or` to combine multiple filters.

An `$and` operator will return results that match all the filters in the list.

Chroma

```
        {
            "metadata_field": {
                <Operator>: <Value>
            }
        },
        {
            "metadata_field": {
                <Operator>: <Value>
            }
        }
    ]
}
```

For example, here we query all records whose `page` metadata field is between 5 and 10:

Python TypeScript Rust

```
collection.query(
    query_texts=["first query", "second query"],
    where={
        "$and": [
            {"page": {"$gte": 5 }},
            {"page": {"$lte": 10 }},
        ]
    }
)
```

An `$or` operator will return results that match any of the filters in the list.

Chroma

```
        {
            "metadata_field": {
                <Operator>: <Value>
            }
        },
        {
            "metadata_field": {
                <Operator>: <Value>
            }
        }
    ]
}
```

For example, here we get all records whose `color` metadata field is `red` or `blue`:

Python TypeScript Rust

```
collection.get(
    where={
        "$or": [
            {"color": "red"},
            {"color": "blue"},
        ]
    }
)
```

Using Inclusion Operators

The following inclusion operators are supported:

- `$in` - a value is in predefined list (string, int, float, bool)
- `$nin` - a value is not in predefined list (string, int, float, bool)

Chroma

```
{
  "metadata_field": {
    "$in": ["value1", "value2", "value3"]
  }
}
```

An `$nin` operator will return results where the metadata attribute is not part of a provided list (or the attribute's key is not present):

Python TypeScript Rust

```
{
  "metadata_field": {
    "$nin": ["value1", "value2", "value3"]
  }
}
```

For example, here we get all records whose `author` metadata field is in a list of possible values:

Python TypeScript Rust

```
collection.get(
    where={
       "author": {"$in": ["Rowling", "Fitzgerald", "Herbert"]}
    }
)
```

Using Array Metadata

Chroma supports storing arrays of values in metadata fields. You can use the `$contains` and `$not_contains` operators to filter records based on whether an array field includes a

Chroma

Metadata arrays can contain strings, integers, floats, or booleans. All elements in an array must be the same type.

Python TypeScript Rust

```python
collection.add(
    ids=["m1", "m2", "m3"],
    embeddings=[[1, 0, 0], [0, 1, 0], [0, 0, 1]],
    metadatas=[
        {"genres": ["action", "comedy"], "year": 2020},
        {"genres": ["drama"], "year": 2021},
        {"genres": ["action", "thriller"], "year": 2022},
    ],
)
```

Filtering with `$contains` and `$not_contains`

Use `$contains` to check if a metadata array includes a specific scalar value, and `$not_contains` to check that it does not.

Chroma

```
    where={"genres": {"$contains": "action"}}
)

# Get all records where genres does NOT contain "action"
collection.get(
    where={"genres": {"$not_contains": "action"}}
)

# Works with integer arrays too
collection.get(
    where={"scores": {"$contains": 20}}
)

# Combine with other filters
collection.get(
    where={
        "$and": [
            {"genres": {"$contains": "action"}},
            {"year": {"$gte": 2021}},
        ]
    }
)
```

Supported Array Types

Type	Python	TypeScript	Rust
String	`["a", "b"]`	`["a", "b"]`	`MetadataValue::StringArray(...)`
Integer	`[1, 2, 3]`	`[1, 2, 3]`	`MetadataValue::IntArray(...)`
Float	`[1.5, 2.5]`	`[1.5, 2.5]`	`MetadataValue::FloatArray(...)`
Boolean	`[true, false]`	`[true, false]`	`MetadataValue::BoolArray(...)`

Constraints:

Chroma

- The `$contains` value must be a scalar that matches the array's element type.

Combining with Document Search

`.get` and `.query` can handle metadata filtering combined with **document search**:

Python TypeScript Rust

```python
collection.query(
    query_texts=["doc10", "thus spake zarathustra", ...],
    n_results=10,
    where={"metadata_field": "is_equal_to_this"},
    where_document={"$contains":"search_string"}
)
```

Was this page helpful? Yes No Suggest edits

Query and Get
‹ Previous

Full Text Search
Next ›

Chroma

Enterprise

Chroma